# PARADIGMA ORIENTADO A OBJETOS
# TRABAJO PRÁCTICO

## Cuatrimestre: 1 – Año: 2026


**UNIVERSIDAD ARGENTINA DE LA EMPRESA**

**FACULTAD DE INGENIERÍA Y CIENCIAS EXACTAS**

**Integrantes del Grupo Nro. : 4 - "MethodApproved"**

| | |
|---|---|
| *Zoilo Guzman* | *LU 1183689* |
| *Franco Valvecchia* | *LU 1113356* |
| *Ledesma Matias* | *LU 1201793* |
| *Cristiani Tiziano* | *LU 1146378* |
| *Salvucci Ramiro* | *LU 1104160* |
| *Nahuel Caballero* | *LU 1155198* |
| *Luca Manuel Schinder* | *LU 1195371* |

**Docente: Hernández, Pablo Eduardo**

## Actividad 1 - Jerarquías y Comportamientos Especializados

## Actividad 2 - El Desafío GRASP y SOLID

**1-** La clase ProcesadorDeVentas viola el Principio de Responsabilidad Única (SRP). La clase pasó a tomar el rol de una clase “maestra”, que se encarga de todas las tareas. Si eventualmente se decidiese modificar algún aspecto de la facturación, los impuestos, o el stock, habría que modificar la misma clase en ambos casos. Eso aumenta las probabilidades de generar incompatibilidades entre el código nuevo y el versionado original. Para resolver el problema, recomendamos particionar la clase en unidades más pequeñas, que permitan compartimentalizar los métodos de manera más coherente; por ejemplo, en la siguiente suma de clases:

- Clase CalculadorImpuestos: Clase encargada de gestionar el cálculo de impuestos y parametrizaciones de dicho proceso.
- Clase ServicioFacturacion: Encargada de generar las facturas e imprimirlas
- Clase HistorialVentas: Focalizada en la persistencia de los datos en la base de datos
- Clase GestorInventario: Clase afin de actualizar el stock de cada producto en base a las ventas

Esta distribución de las responsabilidades permite que la construcción de la solución sea más cohesiva, con cada clase cumpliendo tareas en un encuadre definido. De esta manera el código será más fácil de testear, y las modificaciones parciales de distintos aspectos no impactarán la solución de manera integral.

**2-**
**Experto en Información:** La responsabilidad de calcular el total de una factura debe recaer en la clase Factura dado que, como el patrón indica, la tarea debe asignarse a quien posea la información necesaria para realizarla; la factura conoce los ítems comprados, sus cantidades, y el precio por unidad. Es la única clase de las dos que podrá calcular el total a pagar.

**Creador:** La clase que debe instanciar los objetos LineaDeDetalle debe ser la clase Factura. Esto se debe a que el patrón Creador sugiere que la clase A debe contener a B si contiene, compone, o utiliza a B. Lógicamente, las líneas de detalle se registran en la factura; la factura estará compuesta por dichas líneas y tiene los datos necesarios para inicializarla.

**3-. Miembros de clase (static):**
Por lo visto en la consigna el módulo se encuentra acoplado a una implementación concreta, lo que impide cambiar de tecnología sin modificar el código cliente, es por eso que si mañana el sistema necesita imprimir en PDF o en impresora térmica, hay que modificar ModuloFacturacion. Y eso de por sí viola el DIP, además del principio Abierto/Cerrado (OCP).

¿Cuál sería nuestra solución?

Para resolver este problema se introduce una interfaz llamada **Impresora** que actúa como contrato común. Todas las implementaciones concretas deben cumplir ese contrato, y el módulo de facturación depende únicamente de la interfaz, sin conocer los detalles de ninguna implementación particular.

La dependencia concreta se recibe desde afuera a través del constructor, lo que se conoce como Inyección de Dependencias.

```
INTERFAZ Impresora:
   MÉTODO imprimir(factura): void


CLASE ImpresoraLaser IMPLEMENTA Impresora:
   MÉTODO imprimir(factura):
      MOSTRAR "Imprimiendo en láser: " + factura.getNumero()


CLASE ImpresoraTermica IMPLEMENTA Impresora:
   MÉTODO imprimir(factura):
      MOSTRAR "Imprimiendo en térmica: " + factura.getNumero()


CLASE ModuloFacturacion:
   ATRIBUTO impresora: Impresora

   CONSTRUCTOR(impresora: Impresora):
      this.impresora = impresora

   MÉTODO emitirFactura(factura):
```

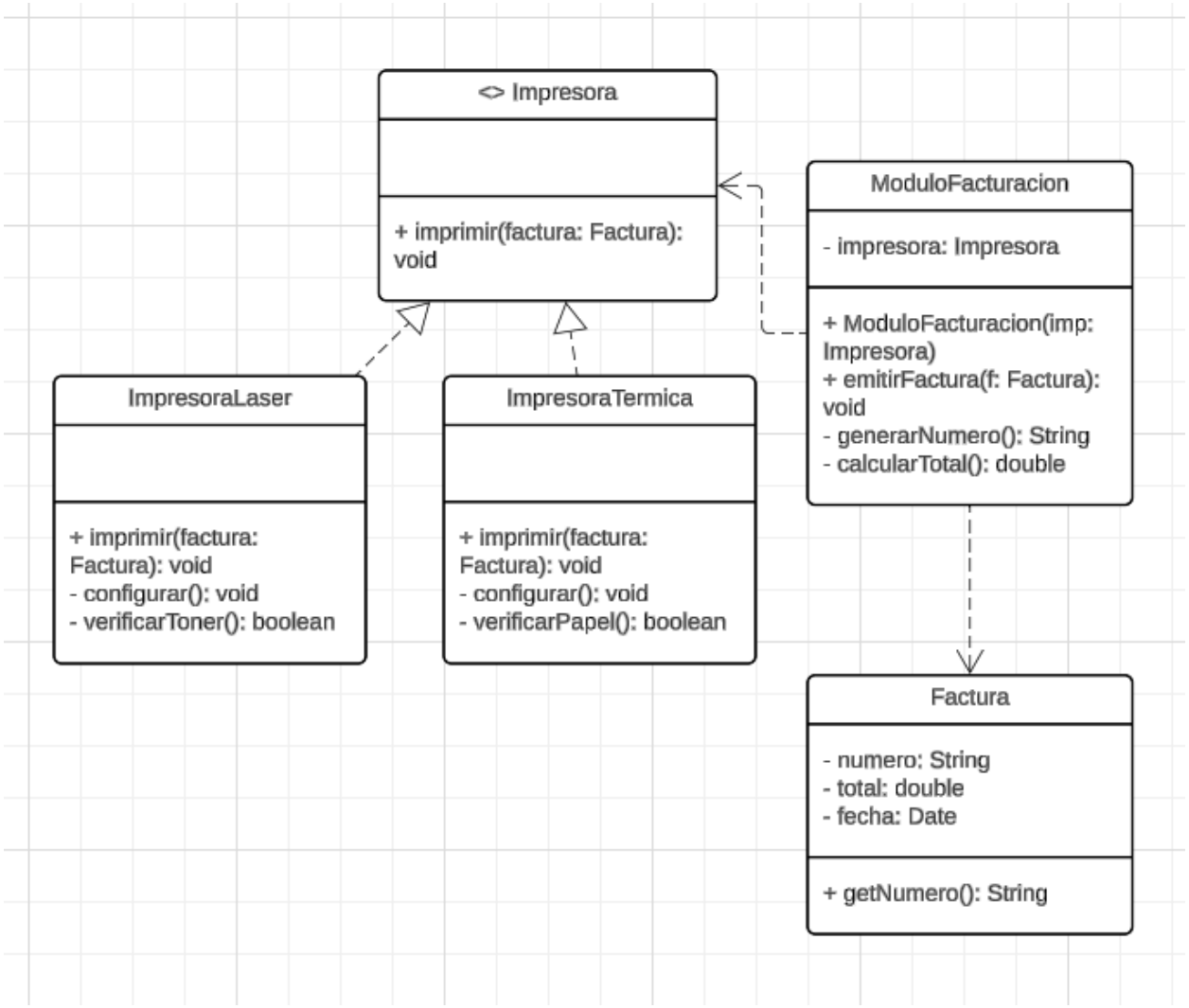
```
    // lógica de generación...
    this.impresora.imprimir(factura)


// Uso — quien instancia decide qué impresora usar
modulo1 = NUEVO ModuloFacturacion(NUEVO ImpresoraLaser())
modulo2 = NUEVO ModuloFacturacion(NUEVO ImpresoraTermica())

modulo1.emitirFactura(factura)    // usa impresora láser
modulo2.emitirFactura(factura)    // usa impresora térmica
```

Al introducir la interfaz **Impresora**, el módulo de facturación deja de conocer los detalles de implementación. Si en el futuro se agrega una nueva tecnología de impresión, alcanza con crear una nueva clase que implemente la interfaz, sin tocar el resto del sistema.

Esto logra **bajo acoplamiento** entre el módulo de alto nivel y los módulos de bajo nivel, y respeta el principio de **Inversión de Dependencias**: ambos dependen de la abstracción, no entre sí.

La inyección por constructor es el mecanismo que hace posible este desacoplamiento, ya que delega al código externo la decisión de qué implementación concreta utilizar en cada contexto.

Diagrama:


<> Impresora
+ imprimir(factura: Factura): void
ModuloFacturacion
- impresora: Impresora
+ ModuloFacturacion(imp: Impresora)
+ emitirFactura(f: Factura): void
- generarNumero(): String
- calcularTotal(): double
ImpresoraLaser
+ imprimir(factura: Factura): void
- configurar(): void
- verificarToner(): boolean
ImpresoraTermica
+ imprimir(factura: Factura): void
- configurar(): void
- verificarPapel(): boolean
Factura
- numero: String
- total: double
- fecha: Date
+ getNumero(): String

**5**

Alta cohesión significa que cada una de las clases tiene una única responsabilidad bien definida, en el diseño original con if y else la clase que calculaba los descuentos tenía baja cohesión ya que conocía la lógica de todos los tipos de cliente. Cualquier tipo de cambio afectaba a todas las clases a la vez. Con el diseño nuevo cada clase trabaja de manera individual y solo sabe lo que le corresponde, por ejemplo al cambiar el descuento de cliente VIP solo se toca ClienteVIP y ninguna otra clase se ve afectada.

En cuanto al acoplamiento significa que las clases dependen entre sí lo menos posible, en el diseño original el código calculaba el precio conocía todos los tipos concretos de cliente y agregar un nuevo cliente obligaba a modificar toda la lógica. Con el nuevo diseño el método mostarPrecio depende únicamente de la interfaz cliente